

Software Requirement Specifications Student Grade Prediction System



Submitted by

Aiza Tanveer (S23BINFT1M01048)

Submitted to

Dr. Rana Nazir Ahmad

**Department of Information Technology
Faculty of Computing**

The Islamia University of Bahawalpur

Meeting Details

Sr No	Details	Date	Supervisor Signature
1.	Discussed the project idea (Student grade Prediction system) and received approval from the supervisor.	17/03/2026	
2.	Reviewed and corrected the introduction, requirements, use cases, and SRS formatting	29/04/2026	
3.	Final SRS approved by supervisor after revisions.	30/04/2026	

Summary

The document of SRS outlines all the functional and non-functional requirements of the Student Grade Prediction System, which is a final year project for the Department of Information Technology at Islamia University of Bahawalpur. In the model, machine learning models are been use is Linear Regression and Decision tree, which predict the students' grade using 3 primary features: class attendance, weekly hours studying, and marks from previous exams. In this document, the purpose of the system, scope of the system, product perspective, kinds of users, literature survey on the system to be used, technology to be implemented, 7 functional requirements in standard format, non-functional requirements, and 6 use cases with the use case diagram on the system level and activity diagram of the critical processes have been explained. The purpose of this system is to help instructors to identify the failing students and to help the failing students to improve their performance.

Table of Contents

1. Introduction.....	5
1.1 Purpose.....	5
1.2 Scope.....	5
1.3 Product Perspective.....	5
1.4 User Characteristics.....	6
1.5 Similar apps and systems/Literature Review.....	6
1.6 proposed Technologies	6
 2 Requirements.....	 8
2.1 Functional Requirements.....	8
2.1.1 User Registration.....	8
2.1.2 User Login.....	8
2.1.3 Enter Student Data.....	9
2.1.4 Grade Prediction.....	9
2.1.5 View Prediction Results.....	9
2.1.6 View Data Analytics.....	10
2.1.7 Manage Prediction Model.....	10
2.2 Non-Functional Requirements.....	11
 3 Use Cases and Flow of Processes.....	 10
3.1 Use Case 1 – Login / Register.....	12
3.2 Use Case 2 – Enter Student Data.....	14
3.3 Use Case 3 – Predict Grade.....	16
3.4 Use Case 4 – View Prediction Results.....	17
3.5 Use Case 5 – View Data Analytics.....	18
3.6 Use Case 6 – Manage Prediction Model.....	18
4 References.....	19

1. Introduction

Student Grade Prediction System is an independent and Web-based academic utility built for the Department of Information Technology, Islamia University, Bahawalpur. The Information Technology department provides undergraduate and post-graduate studies for students pursuing IT and allied disciplines and caters to a few hundred students throughout the academic year. A recurring problem is that faculty want to know which students are most likely to underperform, prior to the examination phase. Through the application of supervised machine learning models on a database of data collected as part of normal university operations, attendance, self-reported study hours, prior exam results, etc. The final predicted grade of each student can be determined. The module being designed will range from the data entry module to the display of the prediction, and it will serve as a tool for assisting the department as an analysis component, in conjunction with official transcripts.

1.1. Purpose

The main objective of the Student Grade Prediction System is to develop an intuitive, data-driven system that can aid educators and students in making well-informed decisions regarding academics. The system is built with the intention of accurately predicting the grades of students with existing machine learning algorithms trained on student performance data. It will enable teachers to anticipate at-risk students early in the semester and render necessary support, and will help students to visualize the impact of their present attendance and study efforts on their final grades, thereby encouraging improvement. The system also assesses and shows the accuracy of the prediction model employed so that the end user is assured of the validity of the outputs generated. Overall, the intention is to produce a convenient, effective, and user-friendly system that can offer academic support to an educator/student, illustrating the application of elementary machine learning.

1.2. Scope

Student Grade Prediction System would have Student Authentication(for Students, Teachers and Admins), Student data entry(for attendance percentage of student, study hours per week and previously obtained marks), Grade prediction(with the help of two algorithms Linear regression and decision tree), display result(predicted grade and confidence of the model), Data analytics dashboard for teachers and admin and model management(for admin to retrain or update machine learning model). The system will be deployed as a web application using Python and Flask with a simple HTML/CSS front end as an option and trained using a public student performance dataset on Kaggle. The following features are out of scope for the project, which is not going to be developed as part of this system. Full LMS functionalities (like course enrollment, submission of assignments, and automated marking), system integration with the university ERP or students' information system, native mobile application, synchronized real-time students' attendance or timetable, and highly advanced security features (like multi-factor authentication, end-to-end encryption).

1.3. Product Perspective

The Student Grade Prediction System is not designed to substitute any existing school management system and is developed as a standalone, self-contained application. The system is

basically a complementary analytical component that can either be deployed on a local machine server or a cloud hosting service. In the design, the system has a Python-based server running the ML model on it, a Flask framework acting as a layer in the application, exposing a few routes to access the application's functions, and a simple HTML/CSS-based frontend that can display the user interface on any current web browser. The prediction component communicates with an already trained Scikit-learn model stored locally on the server. The students and teachers can access the system via a web browser, while an administrator can maintain and manage the users and models via an authenticated admin panel. The system, as it is being developed, can operate as a standalone entity, but it may be updated in a later phase to integrate with the university information systems via a REST API.

1.4. User Characteristics

Three categories of users use the Student Grade Prediction System. Students are end-users of the system. Students register with the system, log into the system, and input their individual student's details such as attendance, study hours per week, past marks, and view their predicted grade and insights. Students need very basic computer and web browser knowledge. Administrator: The administrator is responsible for user account maintenance, system maintenance, and replacing or retraining ML models after new data becomes available. A teacher is someone who uses the system for viewing predictions of their students and to analyze grade distribution using the dashboard (chart-based comparison and correlation between study time and attendance vs marks). Administrators and teachers need moderate to advanced knowledge.

1.5. Similar apps and systems/Literature Review

A few systems and studies have already approached the problem of predicting students' academic success. The systems used by Cortez and Silva (2008) employed decision trees, random forests, and neural networks on student performance data from two secondary schools in Portugal, taking into consideration factors such as family background, hours of study, past marks, and reached promising precision, but without a useful application. Hussain et al. (2018) used k-NN and Naive Bayes classifiers in order to classify student grades; nevertheless, this system was designed for batch prediction and was not intended for student and teacher interaction. Many notebooks found on Kaggle also implemented the Student Performance dataset for grade prediction using Scikit-learn, but remained only demonstrations, without a practical user interface. Enterprise tools such as EAB Navigate and Civitas Learning are capable of complex early-alert and predictive analytics systems, but they are high-level corporate solutions that require an institutional commitment and integration into existing university infrastructure. The presented Student Grade Prediction System can differentiate itself by combining a clear, explainable ML algorithm with a user-friendly interface accessible via a browser without institutional requirements and as a Final Year Project representing an applied ML approach.

1.6. Proposed Technologies

The application will be programmed in Python 3.x since there exists a wide range of machine learning libraries, and it is easily integrable to the web. The machine learning implementation will involve the use of three libraries in particular: Panda for handling the loading of data and manipulating it. For numerical computation, NumPy would be used, and for training the models and for predictions, scikit-learn would be employed. For the training of the models, two algorithms would be chosen and implemented; Linear regression would be used for predicting

the continuous grade score, and a Decision Tree Classifier would be implemented for predicting the categorical grade(A, B, C, D, F). For the web application, Flask, a light-weight Python web micro-framework, would be used. Flask offers request handling and routing without unnecessary overhead from a larger web framework. HTML5 and CSS3 templating would be used in the frontend and rendered through Jinja2, also managed by Flask. The data for the training of the models would be obtained from the Student Performance Data Set, which is available publicly on Kaggle. The application will be developed through the usage of the integrated development environment of Visual Studio Code, along with the use of Jupyter Notebook for rapid experimentation. The final application would be ready to run on any machine operating Windows, Mac OS X, or Linux, with Python 3.x installed. It can be additionally deployed on a remote machine by use of a cloud-based platform such as Heroku or PythonAnywhere.

2. Requirements

The Student Grade Prediction System needs to have at least three users: Students, Teachers, and Admins. Students should be able to create an account and then log in securely and input their academic details to obtain the prediction for the final grade, along with its analytics. Teachers can log in to see the prediction result of the class they teach and the analytics, which will show class grade distribution and behavior trends of attendance and study. Admins should be able to manage the users (create, update, delete) and be able to replace or retrain the machine learning model by providing a new dataset. Input from all users is validated by the system before it gets processed and displayed with appropriate error messages for incorrect input and an accuracy score for the current prediction model, so users can see the accuracy of their result. All predictions should be saved and associated with students for future reference and comparison.

2.1. Functional Requirements

The following functional requirements are defined below in standard format. The requirements are listed, and for each requirement, it directly maps to the functions defined in the requirements overview above.

2.1.1. User Registration

- **Name:** FR001
- **Purpose:** A new user can register and enroll in the system.
- **User(s):** Student, Teacher, Administrator
- **Input:**
 - *Full Name:* Actual name of the user.
 - *Email Address:* A valid, unique email address used as the login identifier.
 - *Password:* Must be at least 8 characters long and include at least one number.
 - *Role:* Selected from: Student, Teacher, or Administrator.
 - *Phone Number:* A working phone number.
- **Output:** New User account is created, User is redirected to the login page with a success message.

2.1.2. User Login

- **Name:** FR002
- **Purpose:** Authenticates a registered user and grants role-based access to the system.
- **User(s):** Student, Teacher, Administrator
- **Input:**
 - *Email Address:* The email address registered during sign-up.

- *Password*: The password set during registration.
- **Output**: The user is authenticated, a session is created, and the user is directed to their role-specific dashboard.

2.1.3. Enter Student Data

- **Name**: FR003
- **Purpose**: Allows a student or teacher to input academic parameters required for grade prediction.
- **User(s)**: Student, Teacher
- **Input**:
 - *Student Name*: Full name of the student whose data is being entered.
 - *Attendance Percentage*: A numeric value between 0 and 100 representing class attendance.
 - *Weekly Study Hours*: A positive numeric value indicating average hours studied per week.
 - *Previous Marks / GPA*: A numeric value representing prior academic performance.
- **Output**: The validated data is stored in the system and submitted to the grade prediction engine.

2.1.4. Grade Prediction

- **Name**: FR004
- **Purpose**: Processes the submitted student data through the active machine learning model to generate a predicted grade.
- **User(s)**: System (automated upon data submission)
- **Input**:
 - *Student Academic Data*: Validated data submitted via FR003 (attendance, study hours, previous marks).
- **Output**: A predicted final grade (A, B, C, D, or F) and a model confidence score are generated and stored.

2.1.5. View Prediction Results

- **Name**: FR005

- **Purpose:** Shows the user the outcome of grade prediction, the influencing factor, and accuracy.
- **User(s):** Student, Teacher
- **Input:**
 - *Prediction Result:* The grade and confidence score generated by FR004.
- **Output:** The user sees predicted final grade, how accurate the active model is, and what the top input factors are.

2.1.6. View Data Analytics

- **Name:** FR006
- **Purpose:** Provides statistical visualizations and insights derived from all stored student data records.
- **User(s):** Teacher, Administrator
- **Input:**
 - *Stored Student Records:* All previously submitted and processed student data entries.
 - *Filter Criteria (optional):* Date range or class/section for filtering the displayed analytics.
- **Output:** Visual charts are rendered including a grade distribution summary, attendance versus grade correlation, and study hours versus grade analysis.

2.1.7. Manage Prediction Model

- **Name:** FR007
- **Purpose:** Enables the administrator to retrain or update the machine learning prediction model with new training data.
- **User(s):** Administrator
- **Input:**
 - *Training Dataset:* A CSV file containing valid student performance records with the required columns.
 - *Algorithm Selection:* Choice of Linear Regression or Decision Tree Classifier.
- **Output:** Finally, the system re-trains the model, tests the accuracy on the test split, saves the newly trained model, and displays the new accuracy metric.

2.2. Non-Functional Requirements

- 1.** Prediction must be generated within 3 seconds of data entry when the system is operating at normal load
- 2.** The user interface must be self-evident and require no more than 5 minutes of onboarding for new, non-expert users.
- 3.** The prediction model should be at least 75% accurate on the hold-out test set.
- 4.** User passwords will be secured with a safe hashing method (bcrypt is the preferred method). User sessions will automatically expire after 30 minutes of inactivity.
- 5.** System will support 500 simultaneous users with no detectable degradation in performance.
- 6.** System will be at least 99% available when it is active during semesters.
- 7.** Code will adhere to PEP8 Python style guides and will contain comments for all non-obvious code lines.
- 8.** The system must be platform independent and should be deployable on Windows, macOS, and Linux using any basic installation of Python 3.x. No other software beyond a standard Python install should be required to deploy and run.

3. Use Cases and Flow of Processes

Use cases provide a formal description of the process flow as dictated by the functional requirements. The use case describes what is going to happen when the use case starts, the post-conditions that hold after successful completion, and the sequence of steps followed, including alternative steps and exceptional cases. The use case diagram showing all the system use cases and how they are associated with the three actors of type Student, Teacher, and Administrator can be seen below.

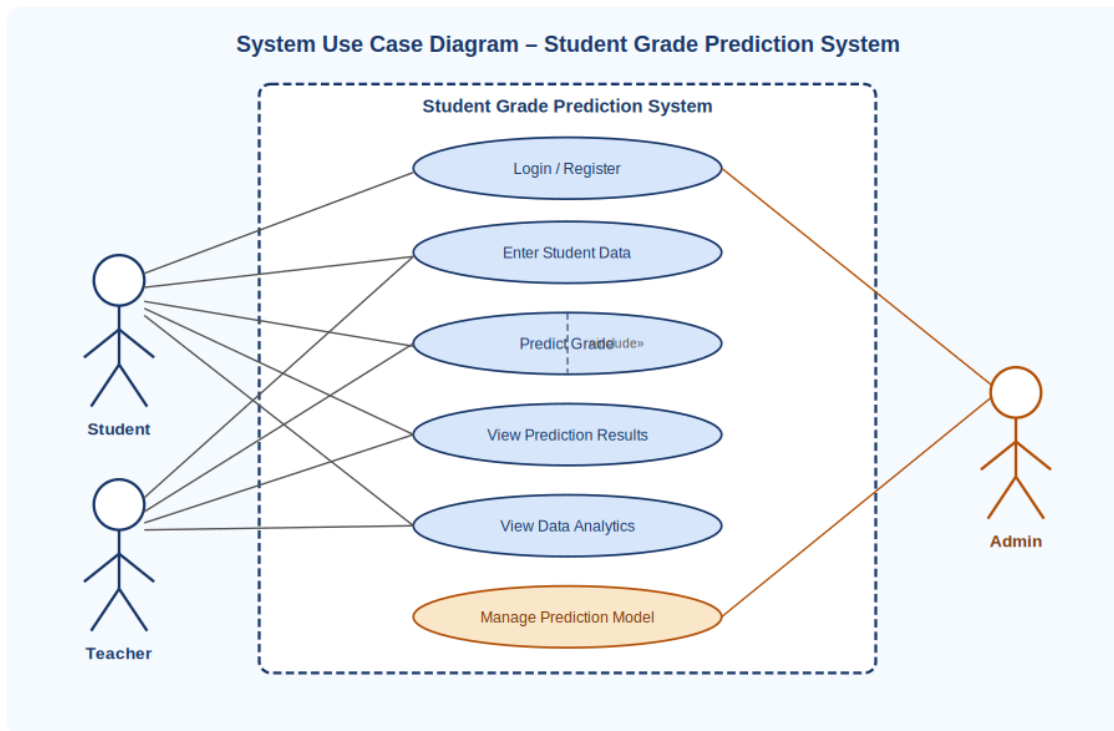


Figure 1: System Level Use Case Diagram

3.1. Use Case 1 – Login / Register

ID	UC001
Name	Login / Register
Description	This use case describes the process by which a new user registers for an account or an existing user logs in to gain access to the system.
Requirement(s)	FR001, FR002
Actor(s)	Student, Teacher, Administrator
Precondition	For registration, the user must not already have an account. For login, the user must be a registered member of the system.

Postcondition	The user is authenticated and redirected to their role-specific dashboard.
Basic Flow	Basic Flow 1. User opens the application and reaches the login page 2. If a new user, the user registers (name, email, password, role, phone #). 3. System checks to make sure the email isn't in the system, the password is long enough, and all fields are filled in. 4. Upon successful validation, the system creates a user and brings the user to the login page with a message saying their account was successfully created. 5. User logs into their account by entering their email and password. 6. System validates login and creates an authenticated session for the user. 7. User is then redirected to their specific role dashboard. Alternative Flow 8. Administrator logs into the admin panel and directly creates a new user account, entering all fields on behalf of the user. Exceptions 9. Invalid or duplicate email during registration: The system displays a descriptive error message and highlights the problematic field. 10. Incorrect credentials during login: The system displays an authentication error without revealing which field is incorrect.

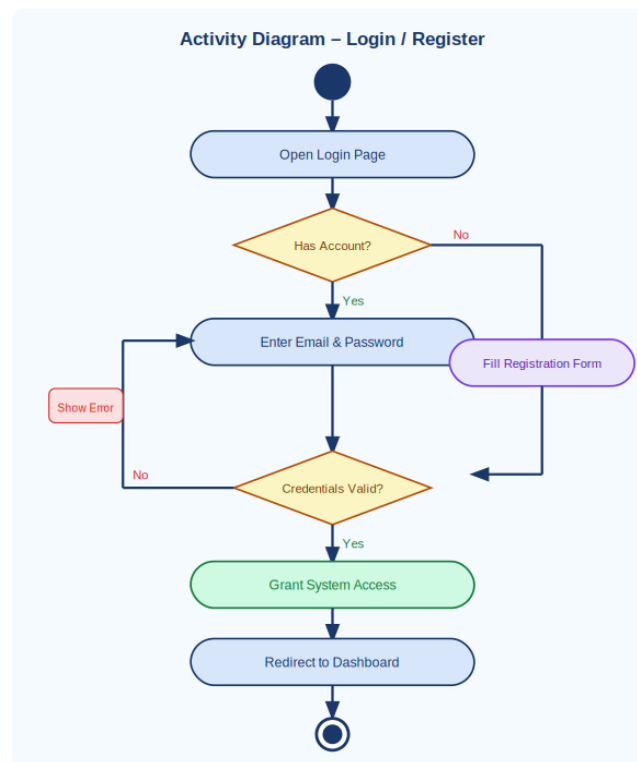


Figure 2: Activity Diagram – Login / Register Process

3.2. Use Case 2 – Enter Student Data

ID	UC002
Name	Enter Student Data
Description	This use case describes the process by which a student or teacher submits academic input data required for grade prediction.
Requirement(s)	FR003
Actor(s)	Student, Teacher
Precondition	The user must be authenticated and logged in to the system.
Postcondition	The student data is validated, stored in the system, and submitted to the grade prediction engine.
Basic Flow	Basic Flow 11. User navigates to the data entry form from the dashboard. 12. User enters the student name, attendance percentage, weekly study hours, and previous marks or GPA. 13. System validates each field: attendance must be between 0 and 100, study hours must be a positive number, and previous marks must be within the valid range. 14. On successful validation, the system stores the data and submits it to the prediction engine (UC003). 15. System displays a confirmation message and triggers the prediction result view. Alternative Flow 16. A teacher enters data on behalf of multiple students by repeating the data entry form for each student. Exceptions 17. Out-of-range values or missing fields: system displays inline validation error messages and prevents submission until all fields are correctly completed.

3.3. Use Case 3 – Predict Grade

ID	UC003
Name	Predict Grade
Description	This use case describes the automated process by which the system applies the active machine learning model to submitted student data and generates a grade prediction.

Requirement(s)	FR003, FR004
Actor(s)	System (automated)
Precondition	Valid student data must have been submitted successfully through UC002.
Postcondition	A predicted grade and model confidence score are generated and stored, linked to the student record.
Basic Flow	<i>Basic Flow</i> 18. System receives the validated student data from UC002. 19. System loads the active pre-trained machine learning model from storage. 20. The model processes the input features (attendance, study hours, previous marks) and computes the predicted grade. 21. System stores the prediction result and confidence score linked to the student record. 22. System triggers the result display view (UC004). <i>Alternative Flow</i> 23. No alternative flow. This use case is fully automated and system-driven. <i>Exceptions</i> 24. If the model file is missing or corrupted, the system displays a maintenance error message and notifies the administrator by logging the event.

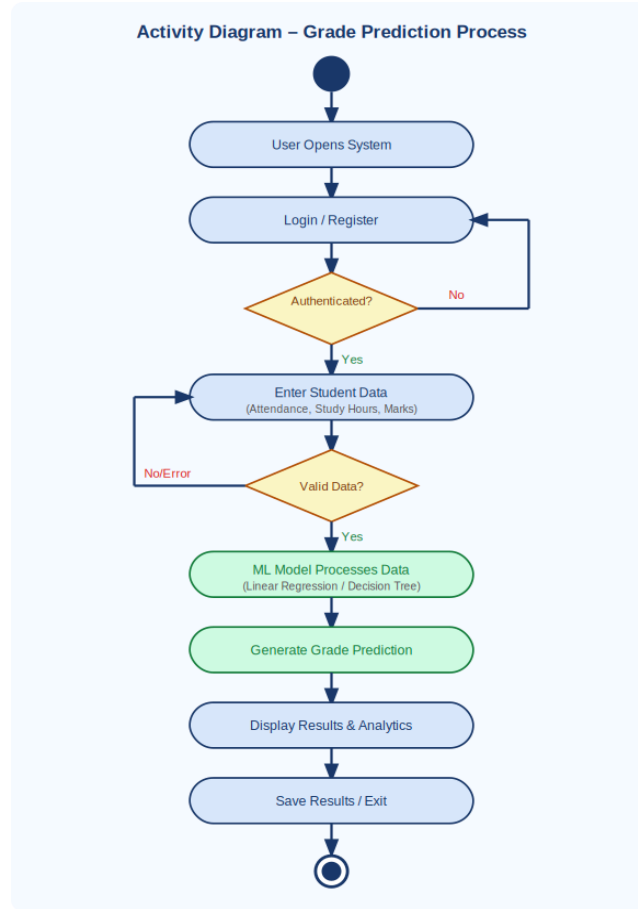


Figure 3: Activity Diagram – Grade Prediction Process

3.4. Use Case 4 – View Prediction Results

ID	UC004
Name	View Prediction Results
Description	This use case describes the process by which a user views the grade prediction outcome, along with contributing factor analysis and model accuracy information.
Requirement(s)	FR004, FR005
Actor(s)	Student, Teacher
Precondition	A grade prediction must have been successfully generated for the student through UC003.
Postcondition	The user has reviewed the predicted grade, confidence score, top contributing factors, and model accuracy.

Basic Flow	Basic Flow 25. System displays the predicted final grade prominently on the results screen. 26. System displays the model accuracy percentage to indicate the reliability of the prediction. 27. System highlights the most influential input factors contributing to the predicted grade. 28. The user may choose to re-enter data for a new prediction or return to the dashboard. Alternative Flow 29. Teacher views prediction results for any student in their class by selecting the student from a class list. Exceptions 30. If no prediction exists for the selected student, the system displays a prompt directing the user to enter student data first.
-------------------	---

3.5. Use Case 5 – View Data Analytics

ID	UC005
Name	View Data Analytics
Description	This use case describes the process by which a teacher or administrator views statistical visualizations and insights generated from stored student data.
Requirement(s)	FR006
Actor(s)	Teacher, Administrator
Precondition	At least one student data record and its prediction result must exist in the system.
Postcondition	The user has viewed the analytics dashboard, including grade distribution, attendance correlation, and study hours analysis.
Basic Flow	Basic Flow 31. User navigates to the Analytics dashboard from the main menu. 32. System retrieves all stored student records and prediction results. 33. System renders a grade distribution summary chart. 34. System renders an attendance versus grade correlation chart. 35. System renders a study hours versus grade analysis chart. 36. The user may apply optional filters, such as date range or class section, to refine the displayed data. Alternative Flow 37. Administrator views analytics across all classes and departments without filtering restrictions. Exceptions

	38. If no student data records exist, the system displays an informational message prompting the user to enter student data before accessing analytics.
--	---

3.6. Use Case 6 – Manage Prediction Model

ID	UC006
Name	Manage Prediction Model
Description	This use case describes the process by which an administrator retrains or updates the machine learning prediction model using a new dataset.
Requirement(s)	FR007
Actor(s)	Administrator
Precondition	The administrator must be authenticated. A valid CSV dataset containing the required columns must be available for upload.
Postcondition	The updated model is trained, evaluated, saved, and deployed. New accuracy metrics are displayed in the system.
Basic Flow	Basic Flow 39. Administrator navigates to the Model Management panel from the admin dashboard. 40. Administrator uploads a new training dataset in CSV format. 41. Administrator selects the desired machine learning algorithm: Linear Regression or Decision Tree. 42. System validates the dataset structure, checking that required columns are present and data types are correct. 43. System splits the data into training and test sets, retrains the model on the training set, and evaluates accuracy on the test set. 44. The new model is saved to server storage and set as the active prediction model. 45. System displays the new model accuracy metric on the admin dashboard. Alternative Flow 46. Administrator rolls back to the previous model version if the new model accuracy is below the acceptable threshold. Exceptions 47. Invalid dataset format or missing required columns: The system displays a descriptive error message with guidance on the expected CSV structure. 48. Insufficient data rows for training: the system alerts the administrator that a minimum number of records is required.

4. References

- [1] Cortez, P., & Silva, A. M. G. (2008). Using data mining to predict secondary school student performance. In A. Brito & J. Teixeira (Eds.), *Proceedings of the 5th Annual Future Business Technology Conference* (pp. 5–12). Porto, Portugal.
- [2] Hussain, S., Dahan, N. A., Ba-Alwi, F. M., & Ribata, N. (2018). Educational data mining and analysis of students' academic performance using WEKA. *Indonesian Journal of Electrical Engineering and Computer Science*, 9(2), 447–459.
- [3] Kaggle. (2023). Students' performance in exams dataset. Retrieved from <https://www.kaggle.com/datasets/spscientist/students-performance-in-exams>
- [4] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [5] Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
- [6] IEEE Computer Society. (1998). IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications. IEEE.